

P2123R0

Extending the Type System to Provide API and ABI Flexibility

Published Proposal, 2020-03-02

This version:

<http://wg21.link/p2123>

Issue Tracking:

[Inline In Spec](#)

Authors:

[Hal Finkel](#) (Argonne National Laboratory)

[Tom Scogland](#) (Lawrence Livermore National Laboratory)

Audience:

SG17

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

For C++, in some collective sense, our community wants the flexibility to change the implementation details and interfaces of types in order to realize performance benefits and otherwise enhance functionality while simultaneously wanting the interface stability necessary to compose software from compiled code created by multiple entities over long time spans. C++ does not currently provide the language-level tools necessary in order to express different points in this flexibility vs. stability space, and this makes it increasingly difficult to evolve the language and its standard library in useful directions. In this paper, we'll explore requirements and a potential solution to this problem.

Table of Contents

1	Introduction
2	Changelog
3	Implementation
4	What Are We Talking About?
5	What Are We Not Talking About?
6	On Costs
7	Some Past Experience
8	Just Recompile!
9	Does This Help With ...
10	Requirements
11	Differences From Existing Features
12	Other Languages
13	How Do I?
14	ODR and Definition Flexibility
15	Proposal
15.1	Interface Tags
15.2	Interface Blocks
15.3	Interface Qualifiers
15.3.1	Interface Additivity
15.3.2	Unresolved Interfaces
15.4	Header <code><stdinterface></code>

§ 1. Introduction

The Application Binary Interface (ABI) in C++ is the architecture-specific protocol, including calling conventions and structure-layout rules, used on a system in order to support separate compilation of C++ code and the composition of programs from the output of those compilation tasks (often in the form of object files, static libraries, and/or shared libraries). In Prague, the committee considered the question of whether or not to take an "ABI break" for the C++23 revision of the standard. While the C++ specification does not specify an ABI, or ABI requirements, directly, our ability to evolve the standard requires consideration of ABI compatibility constraints imposed by the the design of existing shipping C++ ABIs. The results, unfortunately, were fairly inconclusive. A reasonable interpretation of the poll results is that the committee is willing to consider an ABI break at some point in the future, but not right now, and that while performance is an important objective of C++, stability is also important. Moreover, it is unclear exactly what change to facts on the ground would lead to a consensus shift in this regard. This proposal seeks to change the facts on the ground by providing additional language facilities to manage interface stability, including between different revisions of the standard, over time.

Would adopting this proposal essentially imply an ABI break? Yes and no. It's more complicated than that. By turning ABIs and interfaces into first-class facets of the language, code written may, by default, be ABI-incompatible with older C++ code. However, there's an explicit and natural way to use this older code (i.e., to explicitly have access to, and provide new functionality following, the older ABI). This allows C++ programmers to manage ABI transitions and explicitly choose where and how to incur any associated performance penalties.

Note that there is a lot of prior work in this general area, both in the generalized C/C++ ecosystem, and in other programming languages. See, for example, Swift's module stability / library evolution model and Microsoft's Component Object Model (COM).

§ 2. Changelog

- r0: Initial version.

§ 3. Implementation

This proposal has not been implemented.

§ 4. What Are We Talking About?

Before we discuss how we might design language-level facilities to deal with ABI transitions, let's discuss ABI constraints in more detail. The C++ specification does not specify an ABI, or ABI requirements, directly, but there are common implementation techniques that affect how we can evolve the specification, and the implementations themselves, while retaining the ability for a single implementation to serve both code compiled in the context of newer and older C++ standards.

Many of these implementation techniques are designed around supporting C++'s separate-compilation model: An application can be composed, at compile time, from multiple independently-compiled translation units. On many systems, in addition, an application can be composed, during program execution, from multiple independently-compiled translation units linked in the form of shared libraries. Some of these shared libraries are loaded during application startup, and sometimes, the application specifically directs the loading of additional shared libraries serving as "plugins" or similar.

C++ allows some kinds of entities to be defined in multiple translation units in a program so long as their definitions are the same. This leads to our well-known One-Definition Rule (ODR). At an implementation level, to support separate compilation, this means that the implementations of these inline-linkage entities are often emitted multiple times (e.g., into multiple object files, shared libraries) and the linker and/or dynamic loader somehow picks one of these implementation instances to use for a particular program execution. For a conforming program, none of this should be visible. It's important to note that we're not just talking about entities that have an explicit manifestation in the C++ language, but also things, such as virtual-function tables, that are implicitly defined by the C++ code.

In practice, ABIs are composed from different architecture/system-specified pieces, including:

- The C ABI: structure layouts, calling conventions, TLS implementation details, and so on. Might only apply to extern "C" entities, but is often used for many C++ entities as well.
- C++ name-mangling rules (i.e., how to encode the overload-identifying name of a function as a symbol name).
- The representation of C++ primitives (e.g., member pointers).

- The layout of aggregates that don't have direct C analogues (e.g., have base classes, virtual functions).
- The layout of virtual-function tables, RTTI information, and other implementation-internal metadata.
- The method used to implement exception handling.

When discussing this issue, we sometimes end up discussing both:

- The possibility of changing existing types in a way that is ABI-incompatible with existing implementations of those types.
- The possibility of changing/extending the ABI itself in order to improve functionality.

What kinds of changes to a type are ABI-incompatible in this sense? These include:

- Changing the size of the type, the layout of the type (e.g., the order of the members, alignment, adding `[[no_unique_address]]`, modifying base classes, adding the first virtual function).
- Changing the size or contents of the virtual-function table (e.g., changing the order in which virtual functions are defined, adding a new virtual function).
- Changing the return type of a function.

There are some ABI-changing modifications one might make to a class that are not necessarily ABI incompatible, such as changing the type of a function, because an implementation might use the "provide both the new and old symbol" technique to simultaneously serve both old and new clients.

It is also, of course, possible to make incompatible API (i.e., source-level) changes to a type. For example, adding new function overloads, removing a function, and so on.

Changes to the ABI itself, such as changing the calling convention (e.g., how many registers are used for parameter passing, which register holds the `this` pointer), are also part of the conversation around ABI changes and might be something a facility for dealing with ABI changes wishes to address.

It is also important to be clear about whether we intend only to allow different, ABI-incompatible types of the same name to exist in the same program, which is currently often impossible because of inline-linkage definitions and other shared data (e.g., virtual-function tables), or whether we intend to allow for some translation unit to use both of these types simultaneously. It has been suggested that we need to only handle the former case and not the latter. While this may be a helpful step in the right direction, it is not clearly sufficient because this restricts us to a programming model where different parts of the application must use "C" interfaces to communicate across ABI boundaries, including using "C" types to marshal all data across these boundaries (instead of using the C++ types directly).

§ 5. What Are We Not Talking About?

We are not talking about changes to the semantics of existing types. If we want to make an existing function do something different, refrain from doing something, or do something more, with respect to other existing functions or public data members, this kind of ABI-change-handling facility may not help us. These are not ABI changes, but changes in semantics, and semantics changes should often change the names of types regardless of any other considerations.

Thus, while a proposal to handle ABI changes can potentially deal with some kinds of API-level changes, such as removing a function, it cannot, in general, deal with associated semantic changes, such as the ability for the type to correctly implement the removed function.

§ 6. On Costs

Dealing with multiple implementations of library types nearly always imposes some kind of cost. Language-level facilities in this space may, and likely should, provide the programmer with a way to incur / trade off these costs in a way that makes sense for each particular application. A programmer might trade off the costs of:

- Indirection (i.e., performance overhead associated with dynamically-resolved abstraction)
- Specialization (i.e., overheads associated with generating multiple specializations)
- Copying (i.e., the overheads associated with explicitly transitioning between data structures by copying data between one and the other)
- Technical debt (i.e., only using older types in the application, thus preventing the use of newer features)
- Ecosystem isolation (i.e., only using the newest types, perhaps preventing integration with other libraries and services)

§ 7. Some Past Experience

Some C++ standard-library implementations, such as libc++, use C++ inline namespaces and these may succeed at providing a partial strategy for dealing with ABI/API transitions. `std::vector` is really `std::__1::vector`, and so on. With all of the usual caveats about return types not being part of the name mangling on some systems, this can provide unique symbol names when using types from different (inline) namespaces. However:

- The inline namespace technique is not viral: it affects only direct uses of the standard-library types. User types that use standard-library types are not automatically tied to the same versioning scheme, and so, the inline namespace technique does not help user-defined types in the same way.
- On some systems, it doesn't even help standard-library types if they're used only as return types.
- It's not clear how we might use this technique to manage ABI transitions directly.

On this last point, it is important to consider another relevant experience that we have with type aliases (e.g., `intmax_t`). One straightforward suggestion for managing API transitions is to use a new namespace for each revision of the standard. This may work, but leads to a proliferation of vocabulary types (i.e., if `std2` would be bad, this would be `std2` and `std3` and `std4` and so on). A natural suggestion is to allow some of the types from the newer namespaces to be aliases for the types in the older namespaces. However, if this aliasing is implementation defined, or unspecified, it's very difficult to create complete, unambiguous overload sets covering all of these types:

```
void foo(std::cxx20::vector<int> &x);  
  
// Is this needed or ambiguous with the definition above?  
void foo(std::cxx23::vector<int> &x);
```

So for this to work well, we need some adjusted set of rules for overload resolution.

An alternative technique, used by GCC to help manage the C++11 `std::string` transition, is to introduce "viral" ABI tags. The idea is that these tags affect the mangling, but also affect the mangling of types that use the tagged types. It's unclear what should happen if multiple tags are used, but for a single possible tag the high-level description of the behavior seems reasonably simple:

From the GCC manual:

```
abi_tag ("tag", ...)
```

The `abi_tag` attribute can be applied to a function, variable, or `class` declaration. It modifies the mangled name of the entity to incorporate the name, in order to distinguish the function or `class` from an earlier version with a different ABI; perhaps the `class` has changed size, or the function has a different `return` type that is not encoded in the mangled name.

The attribute can also be applied to an `inline namespace`, but does not affect the mangled name of the `namespace`; in this case it is only used for `-Wabi-tag` warnings and automatic tagging of functions and variables. Tagging `inline namespaces` is generally preferable to tagging individual declarations, but the latter is sometimes necessary, such as when only certain members of a `class` need to be tagged.

The argument can be a list of strings of arbitrary length. The strings are sorted on output, so the order of the list is unimportant.

A redeclaration of an entity must not add new ABI tags, since doing so would change the mangled name.

The ABI tags apply to a name, so all instantiations and specializations of `template` have the same tags. The attribute will be ignored if applied to an `explicit` specialization or instantiation.

The `-Wabi-tag` flag enables a warning about a `class` which does not have all ABI tags used by its subobjects and `virtual` functions; for users with code that needs to coexist with an earlier ABI, using this option can help to find all affected types that need to be tagged.

When a type involving an ABI tag is used as the type of a variable or `return` type of a function where that tag is not already present in the signature of the function, the tag is automatically applied to the variable or function. `-Wabi-tag` also warns about this situation; this warning can be avoided by explicitly tagging the variable or function or moving it into a tagged `inline namespace`.

A couple of notes: First, care is required that forward-declared types are properly annotated:


```
struct A;

// We need to mangle the name of A here into the name of the function, so v
// need to know if it has an ABI tag now.
void f(std::unique_ptr<A>);
```

Second, to maintain ODR-provided invariants, we may also need to tag inline-linkage entities based everything used in their definitions, not just those types that appear on the interface. In this example (provided by Richard Smith):

```
inline const char *getThing() {
    static std::string thing = getenv("thing");
    return thing.c_str();
}
```

even if we mangle the name of the static variable using the ABI identifier, it's true that the C++23-compiled `getThing` will return a different pointer than the C++20-compiled `getThing` (and, if we don't always select a consistent implementation of the function matching the language mode of the caller, a different pointer might be returned depending on whether or not any particular call is inlined).

This tag-based mechanism does not address the problem of allowing the user to name both types in the same translation unit.

§ 8. Just Recompile!

We note that some users of C++ can simply recompile all of their code as desired, and thus, these kinds of ABI considerations are not relevant to them. Not all users can, however. Some users can recompile their code as desired, but depend on system libraries and/or system interfaces provided only using some older ABI (e.g. the BeOS API, which is native C++, was like this). Some users can recompile their code but depend on third-party libraries that they cannot recompile. Some users can recompile their code, but their code is a plugin to a third-party application with a fixed ABI. Some users can recompile their code, but their code must load plugins that use the older ABI. Some users can recompile their code, but doing so along with a language-version upgrade is a slow, expensive process if it must happen synchronously across the entire organization.

In short, many users can recompile their code. If they couldn't, they don't care about using the latest C++ features. However, adoption will be increased if users can recompile their code to use the latest

C++ without requiring the same for all transitive dependencies.

§ 9. Does This Help With ...

People have asked whether this proposal would help with some specific use cases that people have had in mind, including:

- `std::regex`

Whether or not this might help with `std::regex` depends greatly on how we might want to change the specification and/or implementations. Presumably, we would want a better (more efficient) way to handle localized character classification, better algorithms and automata optimizations, and so on. We might also need to significantly change the interface to accomplish these goals, however, in which case none of this is particularly relevant (because we'll need to change much of the code using the class regardless and/or just name the new version something else). However, if we restrict this problem to looking only at the problem of updating the algorithms in inline-linkage functions, and potentially, data-structure changes that can be made in a purely-additive way, then yes, this proposal should help that kind of transition.

- `std::unordered_map`

This, again, is going to depend on what kinds of changes we desire to `unordered_map`. If we're trying to allow more implementation freedom, for example, to allow for different kinds of open-addressing strategies, by loosening our iterator-stability guarantees, then that's a semantic change that cannot necessarily be addressed by the proposed facility. If, on the other hand, we're talking about something like changing an implementation from using power-of-two bucket counts to prime bucket counts (or Fibonacci or whatever), then the answer is: maybe. It depends on how deeply the current choice is embedded in the design of the current data structure. If the implementation works with any bucket sizes, it's just that we want to change how the defaults are selected, then this will help. If we can't provide a new implementation that will also work with inlined member functions of the old implementation, then this may not help.

- `std::unique_ptr`

An interesting case because, if the primary desire is to effectively modify the calling convention to prevent passing a pointer to the pointer, this requires a kind of semantic change. For background, see CppCon 2019 talk by Chandler Carruth: "There Are No Zero-cost Abstractions". Because `unique_ptr` has a non-trivial destructor (and constructor) it is passed using the general convention for non-trivial aggregates, thus by pointer. This is inefficient for this kind of wrapper type but, for ABIs where the temporaries are constructed in the caller, ensures a unique value of the this pointer in the constructor

and destructor. We could change this, but that would require making a new kind of object (maybe by leveraging our `[[no_unique_address]]` attribute in a new way). In any case, this proposed facility would help with this transition, as would any facility allowing the implementation to give a new mangling scheme to the `unique_ptr` with these new semantics.

§ 10. Requirements

Some requirements for a facility to help ABI transitions:

1. C++ must default to providing flexibility over interface stability. If the programmer does nothing explicit, then they should get the highest possible performance (relative to that provided by other aspects of this proposal). This includes uses of standard-library types and functions.
2. C++ must provide a way for types to provide interfaces that prioritize stability over flexibility. This must allow code written using a newer version of C++ to call functions in code written in an ABI-incompatible older version of C++, and code written in an older version of C++ to call functions in code written in an ABI-incompatible newer version of C++. It must be possible to implement this feature such that the ABI of pre-C++-23 code can be selected as required.
3. This facility must be generalized, and not specific to different C++ versions. We recognize that the C++ standard library is not the only library with ABI concerns.
4. This facility should work naturally with modules and should strive to not require viral annotations while retaining the ability to make expensive operations explicitly visible to readers of the code.

This proposal certainly introduces new concepts (colloquial) into the C++ language, but aims to do so in a way that integrates naturally and provides behaviors akin to existing aspects of C++ functionality. It does this by:

1. Fully integrating into the type system. This allows existing features such as template specialization and overloading to provide a lot of the necessary semantics.
2. Fully integrating into C++'s name-lookup paradigm, thus making it easier to reason about how names work.

In addition, we recognize that we cannot predict the future, but we can require certain knowledge of the past, and so it must be the responsibility of new code to provide interfaces compatible with older users (and not the other way around).

It's also important to consider the kinds of changes that the stable interfaces should be resilient against because this has an effect on how the implementation is designed. Some questions are:

1. Does stability imply a lack of inlining? Note that there is a potential separation here between inline linkage and actually allowing compiler inlining. Allowing inlining potentially requires "virtual" data-member access (i.e., making the set of internal data members used by these inline functions part of the abstract, stable interface).
2. Does stability imply only additive changes? Adding new functions to the interface? Adding new data members? How about changing the type of existing members? Type changes that change the size vs. type changes that don't change the size? Replacing a data member with a function of other data members?

§ 11. Differences From Existing Features

Upon reading this proposal, you might object that what is being proposed here seems similar to a number of features that we already have in C++. We already have virtual functions, inline namespaces, and so on. Moreover, many of these features are used in practice specifically to support some of the use cases discussed here. Why do we need something new?

Let's try to imagine the best that we could do with existing features if we wanted to update the standard library in a somewhat analogous way to what is effectively proposed here. First, like `libc++` does, we would put all standard-library classes into an inline namespace. For future C++ revisions, implementations would make the namespace containing the previous implementation non-inline, and create a new inline namespace for the new versions of the standard library. This would enable simultaneous access to both the old and new standard library within the same application; technically speaking, changing the namespace definitions like this is an ODR violation, but a benign one in practice, and we could also make this okay with some kind of explicit wording. However, that doesn't address the problem of being able to write code that uses standard-library types on public library interfaces and have it be resilient to standard-library updates. In fact, if we just follow this inline-namespace approach, we've made the problem worse, by forcing all of these interfaces to explicitly traffic in C++-revision-specific standard-library types and, thus, add a lot of manual copying/conversion around all of this public-interface-using code.

Our existing tool for addressing this next problem is to introduce a class hierarchy and make the public interface mostly, or entirely, virtual. If we had such a class hierarchy, we could use these types on public interfaces and, by doing so, gain resilience to functionality additions within the standard-library classes. However, we can't do this for many of the standard-library classes themselves, such as containers, as this would have serious negative performance implications: any time we had a pointer or reference to any of these types, the compiler needs to assume that it might be dealing with some derived class, and none of the functions would be inlined. Moreover, C++ doesn't have a way to say, "X is a pointer to some type T which has virtual functions, but I know the pointed-to object's type is

exactly T and not some derived type." This all implies that our class hierarchy with virtual functions must be a separate hierarchy of wrapper types: A wrapper type for each C++23 standard-library class, a derived wrapper type for each C++26 standard-library class, and so on, with essentially all-virtual interfaces mirroring those of the types wrapped by each.

This has a few problems. First, when using the wrapper types, the fact that all of the functions are virtual means that essentially nothing is inlined, and this leads to suboptimal performance. Also, standard-library types have plenty of templated member functions, and these can't be virtual regardless. In some sense, this implies a somewhat-better solution: Our class hierarchy of wrapper types become friends of the types they wrap, and the interface is the same as the wrapped class, except that virtual functions are used to access the type's data members.

So this leaves us with a library design that looks something like this:

```

#include <utility>

namespace std {
namespace cxx20 {

namespace interface {

    template <typename T>
    class thing;

    template <typename T>
    class thing_wrapper;

}

template <typename T>
class thing {
protected:
    T x, y, z;

    friend class interface::thing_wrapper<T>;

public:
    thing(const T &x, const T &y, const T &z)
        : x(x), y(y), z(z) { }

    T sum() const {
        return x + y + z;
    }
};

namespace interface {

    template <typename T>
    class thing {
protected:
        /*[[object_lifetime_invariant_reference]]*/
        virtual T& get_x() = 0;
        /*[[object_lifetime_invariant_reference]]*/
        virtual const T& get_x() const = 0;

        /*[[object_lifetime_invariant_reference]]*/
        virtual T& get_y() = 0;
    };
}

```

```

/*[[object_lifetime_invariant_reference]]*/
virtual const T& get_y() const = 0;

/*[[object_lifetime_invariant_reference]]*/
virtual T& get_z() = 0;
/*[[object_lifetime_invariant_reference]]*/
virtual const T& get_z() const = 0;

public:
    T sum() const {
        return get_x() + get_y() + get_z();
    }

    virtual operator cxx20::thing<T> &() = 0;
    virtual operator const cxx20::thing<T> &() const = 0;
};

template <typename T>
class thing_wrapper : public thing<T> {
    cxx20::thing<T> U0; // The underlying object.

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_x() { return U0.x; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_x() const { return U0.x; }

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_y() { return U0.y; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_y() const { return U0.y; }

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_z() { return U0.z; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_z() const { return U0.z; }

public:
    thing_wrapper(const T &x, const T &y, const T &z)
        : U0(x, y, z) { }

    thing_wrapper(const cxx20::thing<T> &U) : U0(U) { }

    thing_wrapper(cxx20::thing<T> &&U) : U0(std::move(U)) { }

```

```

        virtual operator cxx20::thing<T> &() override { return U0; }
        virtual operator const cxx20::thing<T> &() const override { return U0; }
    };
}

} // namespace cxx20

inline namespace cxx23 {

namespace interface {
    template <typename T>
    class thing;

    template <typename T>
    class thing_wrapper;
}

template <typename T>
class thing {
protected:
    T x, y, z, w;

    friend class interface::thing_wrapper<T>;

public:
    thing(const T &x, const T &y, const T &z, const T &w = T(0))
        : x(x), y(y), z(z), w(w) { }

    T sum() const {
        return x + y + z + w;
    }
};

namespace interface {

    template <typename T>
    class thing : public cxx20::interface::thing<T> {
    protected:
        /*[[object_lifetime_invariant_reference]]*/
        virtual T& get_w() = 0;
        /*[[object_lifetime_invariant_reference]]*/
        virtual const T& get_w() const = 0;
    };
}

```



```

public:
    T sum() const {
        return this->get_x() + this->get_y() + this->get_z() + get_w();
    }

    virtual operator cxx23::thing<T> &() = 0;
    virtual operator const cxx23::thing<T> &() const = 0;
};

template <typename T>
class thing_wrapper : public thing<T> {
protected:
    cxx23::thing<T> U0; // The underlying object.

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_x() { return U0.x; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_x() const override { return U0.x; }

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_y() { return U0.y; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_y() const override { return U0.y; }

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_z() { return U0.z; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_z() const override { return U0.z; }

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_w() override { return U0.w; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_w() const override { return U0.w; }

public:
    thing_wrapper(const T &x, const T &y, const T &z, const T &w = T(0))
        : U0(x, y, z, w) { }

    thing_wrapper(const cxx23::thing<T> &0) : U0(0) { }

    thing_wrapper(cxx23::thing<T> &&0) : U0(std::move(0)) { }

```

```

        virtual operator cxx23::thing<T> &() override { return U0; }
        virtual operator const cxx23::thing<T> &() const override { return U0;
    };

}

} // inline namespace cxx23
} // namespace std

```

Now you might object that some of the code duplication could be removed by using a CRTP approach and/or other refactoring, and that's likely correct. You might also object that, even if we had the "object_lifetime_invariant_reference" attribute to tell the optimizer that the function always returned a reference to the same object, that still wouldn't be optimal because it would be better to return object-lifetime-invariant object offsets instead, and that's also likely correct. One way or another, however, the code pattern needed here is both heavy on boilerplate and, even with that, performance suboptimal compared to what a compiler might be able to generate for a language feature with similar semantics.

Most importantly, however, there's a design flaw in this library structure. Consider this function that someone may need to write:

```

void resilient_abi(cxx20::interface::thing<int> *);

void call_resilient_abi(std::thing<int> *t) {
    // Assuming for simplicity that we know that t is not nullptr...

    // Should we use the copy or the move constructor of thing_wrapper here?
    std::interface::thing_wrapper<int> *w = new std::interface::thing_wrapper
    resilient_abi(w);

    // Hrmm... should I delete w now? Should I delete t?
}

```

The problem here is that, if I'm wrapping an API that takes an object by pointer or reference, the relevant lifetime and ownership semantics of the pointer/reference parameters are unclear from the type system alone. The API usage here requires deep knowledge, and while in some sense this is okay (it is normally important to understand the behavior of the functions that you call), it also makes it more difficult to integrate use of these interfaces into large code bases. It's not possible for tooling to wrap interfaces like this in a generic way. What we really want is some way to tie the lifetime of *w* to the lifetime of *t* (in the example above), and we don't have any way to do that within this wrapper function.

A solution to this problem is clear, but unfortunate in its own ways. We cannot make the underlying types unconditionally have the virtual functions because that would both unconditionally increase the object size (which breaks one of our requirements), and because it's then not possible to provide both the member functions that use of the virtual data-member access and those that don't (with the same names). An alternative is to create two different variants of each type, one normal one, and one also holds a wrapper object of itself. We can make an interface like this:

```

namespace std {
namespace cxx20 {

namespace interface {

    template <typename T>
    class thing;

    template <typename T>
    class thing_wrapper;

}

template <typename T>
class thing_unwrappable {
protected:
    T x, y, z;

    friend class interface::thing_wrapper<T>;

public:
    thing_unwrappable(const T &x, const T &y, const T &z)
        : x(x), y(y), z(z) { }

    T sum() const {
        return x + y + z;
    }
};

template <typename T>
class thing : public thing_unwrappable<T> {
protected:
    interface::thing_wrapper<T> *wpr;

public:
    thing(const T &x, const T &y, const T &z);
    thing(const thing &P);
    interface::thing_wrapper<T> *wrapper() const { return wpr; }
    ~thing() { delete wpr; }
};

namespace interface {

```

```

template <typename T>
class thing {
protected:
    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_x() = 0;
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_x() const = 0;

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_y() = 0;
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_y() const = 0;

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_z() = 0;
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_z() const = 0;

public:
    virtual ~thing() { }

    T sum() const {
        return get_x() + get_y() + get_z();
    }

    virtual operator cxx20::thing<T> &() = 0;
    virtual operator const cxx20::thing<T> &() const = 0;
};

```

```

template <typename T>
class thing_wrapper : public thing<T> {
    cxx20::thing<T> *U0; // The underlying object.

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_x() { return U0->x; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_x() const { return U0->x; }

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_y() { return U0->y; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_y() const { return U0->y; }
}

```

```

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_z() { return U0->z; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_z() const { return U0->z; }

public:
    ~thing_wrapper() {
        if (U0) {
            U0->release_wrapper();
            delete U0;
        }
    }

    thing_wrapper(const cxx20::thing<T> *0) : U0(0) { }

    virtual operator cxx20::thing<T> &() override { return *U0; }
    virtual operator const cxx20::thing<T> &() const override { return *U0; }
};
} // namespace interface

template <typename T>
thing<T>::thing(const T &x, const T &y, const T &z)
    : thing_unwrappable<T>(x, y, z),
      wpr(new interface::thing_wrapper<T>(this)) { }
template <typename T>
thing<T>::thing(const thing &P)
    : thing_unwrappable<T>(P),
      wpr(new interface::thing_wrapper<T>(this)) { }

} // namespace cxx20

inline namespace cxx23 {

namespace interface {
    template <typename T>
    class thing;

    template <typename T>
    class thing_wrapper;
}

template <typename T>
class thing_unwrappable {

```

```

protected:
    T x, y, z, w;

    friend class interface::thing_wrapper<T>;

public:
    thing_unwrappable(const T &x, const T &y, const T &z, const T &w = T(0))
        : x(x), y(y), z(z), w(w) { }

    T sum() const {
        return x + y + z + w;
    }
};

template <typename T>
class thing : public thing_unwrappable<T> {
protected:
    interface::thing_wrapper<T> *wpr;

public:
    thing(const T &x, const T &y, const T &z, const T &w = T(0));
    thing(const thing &p);
    interface::thing_wrapper<T> *wrapper() const { return wpr; }
    ~thing() { delete wpr; }
};

namespace interface {

    template <typename T>
    class thing : public cxx20::interface::thing<T> {
protected:
        /*[[object_lifetime_invariant_reference]]*/
        virtual T& get_w() = 0;
        /*[[object_lifetime_invariant_reference]]*/
        virtual const T& get_w() const = 0;

public:
        T sum() const {
            return this->get_x() + this->get_y() + this->get_z() + get_w();
        }

        virtual operator cxx23::thing<T> &() = 0;
        virtual operator const cxx23::thing<T> &() const = 0;
    };
}

```

```

};

template <typename T>
class thing_wrapper : public thing<T> {
protected:
    cxx23::thing<T> U0; // The underlying object.

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_x() { return U0->x; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_x() const override { return U0->x; }

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_y() { return U0->y; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_y() const override { return U0->y; }

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_z() { return U0->z; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_z() const override { return U0->z; }

    /*[[object_lifetime_invariant_reference]]*/
    virtual T& get_w() override { return U0->w; }
    /*[[object_lifetime_invariant_reference]]*/
    virtual const T& get_w() const override { return U0->w; }

public:
    ~thing_wrapper() {
        if (U0) {
            U0->release_wrapper();
            delete U0;
        }
    }

    thing_wrapper(const cxx23::thing<T> *U0) : U0(U0) { }

    virtual operator cxx23::thing<T> &() override { return *U0; }
    virtual operator const cxx23::thing<T> &() const override { return *U0; }
};

} // namespace interface

```



```

template <typename T>
thing<T>::thing(const T &x, const T &y, const T &z, const T &w)
    : thing_unwrappable<T>(x, y, z, w),
      wpr(new interface::thing_wrapper<T>(this)) { }
template <typename T>
thing<T>::thing(const thing &P)
    : thing_unwrappable<T>(P),
      wpr(new interface::thing_wrapper<T>(this)) { }

} // inline namespace cxx23
} // namespace std

```

And, now, the user can choose `std::thing`, or `std::thing_unwrappable`, depending on whether or not the user wants to use the type with the ABI-resilient interfaces (which use the wrapper types). The public library interfaces across which we need to be resilient to ABI changes, we can use the wrapper types. The lifetime of the wrapper object is now tied to that of the underlying object. However, because we need to be able to use the delete operator with the wrappers and have that delete the underlying object (and also delete the underlying object and have it delete the wrapper), we're stuck needing two independent objects with dynamic allocation and pointers that would not be necessary if the wrapper were a proper subobject of the object being wrapped.

To goal of this proposal is to give us the benefit of this kind of library structure, but without needing separate object allocations, and without the boilerplate and related usage complexity.

As a language feature, we also have more freedom to provide transparent proxies via the creation of "fat pointers" (i.e., the feature can be implemented by bundling (a pointer to) the metadata associated with the interface abstraction with the pointer to the object itself). This allows the wrapped objects to suffer from no additional size (i.e., no vtable-like pointer needs to be stored in each object). At the time when an unresolved interface pointer is assigned, either the source is another such pointer, in which case the metadata can be copied, or we know the concrete type of the object and the compiler can cause the correct metadata to be stored.

With this proposal, the above library type looks like this:

```

namespace std {
    interface cxx20;
    interface cxx23 : cxx20;

    template <typename T>
    class point {
        interface(cxx20) {
            T x, y, z;
        }

        interface(cxx23) T w;

    public:
        interface(cxx20) {
            point(const T &x, const T &y, const T &z)
                : x(x), y(y), z(z) { }

            T sum() const {
                return x + y + z;
            }
        }

        interface(cxx23) {
            point(const T &x, const T &y, const T &z, const T &w = T(0))
                : x(x), y(y), z(z), w(w) { }

            T sum() const {
                return x + y + z + w;
            }
        }
    };
} // namespace std

```

Now we can use the type in a variety of ways:

```

// A pointer to: interface(<implementation-defined-default>) std::point<int>
std::point<int> *x;

// Same as above.
interface() std::point<int> *x;

// A pointer to: The C++20 std::point<int>.
interface(std::cxx20) std::point<int> *x;

// A pointer to: The C++23 std::point<int>.
// This is usable with an interface taking the C++23-or-later type below.
// The object itself might be a little larger than necessary,
// but should not otherwise have other performance overhead.
interface(std::cxx23) std::point<int> *x;

// A pointer to: The C++23 or later std::point<int>.
// Use of this object will involve some indirect-access overheads.
interface(std::cxx23+) std::point<int> *x;

```

§ 12. Other Languages

Many other languages following this fat-pointer-on-interfaces technique, including:

- Go: go's interface types are runtime generated fat pointers containing a pointer to the object and a pointer to a fixed-offset table of function pointers that can be referenced just like a vtable in the receiver, but are generated by a linear search of a sorted list of function signatures by name (literally by string comparisons, at runtime). It also uses a fat pointer setup for slices. The combination allows for almost complete decoupling of ABI between translation units that communicate only through interface types.
- Swift: Swift uses vtables in all objects by default, and **virtual** methods actually fall back on a runtime message resolution like Objective-C uses, allowing runtime addition and proxying of methods as well as some interesting resilience and ABI stability enhancements on otherwise plain objects. Where fat pointers come in, in this case implemented by passing an extra implicit table argument in addition to an object, is Protocol objects and APIs. A function that takes a protocol object gets a fixed-offset table of pointers to the methods, and possibly data, that are listed in the protocol specific to the object passed in. This is part of how they allow interesting things like static calls to methods added at runtime in the same TU, and the implementation of protocols using those methods for a type in dynamically loaded code.

- Rust: Types are static by default, generics work essentially like templates. Fat pointers are used in a couple of different ways, first they're used for references to runtime-sized slice types. More significantly, they're used to implement trait objects. This is not quite like Go's setup in that the object has to have an explicit implementation of the trait, which allows the compiler to generate the table rather than having to do it at runtime on first use, but the usage is much the same. It allows an API writer to select either entirely static replicated implementation with generics and trait bounds or the indirected trait object that has some runtime overhead but will work with any type that implements the protocol now or in the future.

§ 13. How Do I?

In this section, we'll walk through a number of use cases and discuss how they'll work with this proposal...

- I have code compiled for C++20 that uses modules. How can I use it in C++23 code?

```
import SomeCXX20Code;
// Now use the exported things as you would expect.
```

The implementation is allowed to automatically assign the cxx20 interface tag to all of the things in the module (and although the standard may not be able to mandate it, it should do so).

- I have code compiled for C++20 that does not use modules. How can I use it in C++23 code?

```
#include <stdinterface>
interface(std::cxx20) {
#include <SomeCXX20Code.h>
}
// Now use the included things as you would expect.
```

The interface block will implicitly assign the cxx20 interface tag to everything in the included file, and since nothing in that code uses interfaces, everything should work naturally.

- Okay, it can't be that simple. What's the catch?

The catch is that types from the standard library that are used with that C++20 code are not the same types as the standard-library types used in the rest of your C++23 code. So this won't work...

```
#include <stdinterface>
interface(std::cxx20) {
void foo(std::vector<int> &v);
}
...
std::vector<int> v;
foo(v); // illformed; no conversion
// from interface(std::cxx23) std::vector<int>& to
//   interface(std::cxx20) std::vector<int>&
```

But the point here is that:

1. Both the C++20 and C++23 types can simultaneously coexist in your program (they're different types in every respect - no ODR or linking problems). So you might need to make copies on interfaces here, but this is all well-defined C++ code and the programmers can figure out the best way to handle this for their applications.
 2. This is a one-time transition, and in the future we can use better facilities provided to manage these kinds of transitions.
- Given that C++20 and C++23 standard-library types are different types, can I provide different function overloads for them?

Yes, they're different types. That means that you can provide different overloads for them, different template specializations for them, and so on.

```
#include <stdinterface>
...
void foo(interface(std::cxx20) std::vector<int> &x) { ... }
void foo(interface(std::cxx23) std::vector<int> &y) { ... }
```

but to be resilient against future changes, you might write:

```
#include <stdinterface>
...
void foo(interface(std::cxx20) std::vector<int> &x) { ... }
void foo(interface(std::cxx23+) std::vector<int> &y) { ... }
```

and now the second overload will handle all versions of the standard in the future. There will be some performance overhead as we've now essentially made all member functions on `y` virtual, although the

compiler is certainly free to specialize for the case where the object is of type `std::cxx23::vector` itself - and the programmer can also explicitly specialize by using `dynamic_cast` in the definition of the function (although I wouldn't recommend it, unless the programmer also arranges for some way to test the non-specialized code).

Template instantiation and specialization also work as they usually do:

```
template <typename T>
unsigned foo() { return sizeof(T); }

...
// This returns whatever it did in C++20 for std::vector<int>, probably 24.
foo<interface(std::cxx20) std::vector<int>>>();

// This returns whatever it should now in C++23, probably also 24.
foo<interface(std::cxx23) std::vector<int>>>();
foo<std::vector<int>>>(); // if cxx23 is the default, this is the same as at
```

Note: On the design space here: it is tempting to think of objects with unresolved interface tags as real (complete) "proxy objects" with sizes and so on. There are several alternatives here that don't really seem to work. First, we could have proxy objects as real objects that are first-class values. The problem here comes from functions that want the proxy itself by pointer, as automatic wrapping would require heap allocating the proxy, and then it would need to live for an indefinite period of time (its lifetime should be tied to that of the original object, but we have no good way to do that). An alternative would be to make the user explicitly manage the lifetime of the proxy object, but the ergonomics are not good (and the user of the interface in question might also not own the object or have any way to tie the lifetime of the newly-allocated proxy object to the original). We could say that, somewhat like references, taking the address of the proxy object yields the underlying object, so we never pass pointers to the proxy objects (they're only used for pass by value), but then the size of the proxy object would be inconsistent with the size of the storage represented by the memory at the taken address.

- How can the standard library write a class that provides it's existing external interface to C++20 clients, and a new interface to C++23 clients?

```

#include <stdinterface>
...
struct point {
    interface(std::cxx20) {
        int x, y, z;
        interface(std::cxx23) int w;

        int get_x() const { return x; }

        int get_w() const interface(std::cxx23) { return w; }

        interface(std::cxx23) {
            bool only_for_cxx23() { return true; }
        }
    }
};

// sizeof(interface(std::cxx20) point) == 12.
// sizeof(interface(std::cxx23) point) == 16;

```

This class provides an existing interface to C++20 clients, using its exiting ABI, while simultaneously providing an expanded interface to C++23 clients.

- How can I (not the standard library) write a class that provides it's existing external interface to C++20 clients, and a new interface to C++23 clients?

You do this in much the same way as the standard library, but you only need to consider cxx20 to be special as far as the ABI goes. Going forward, you likely want to decouple your interface updating strategy from that of the C++ standard.

```

#include <stdinterface>
...
interface v2 : std::cxx20;
interface v3 : v2;
...
struct point {
    interface(std::cxx20) {
        int x, y, z;
        interface(v2) int w;

        int get_x() const { return x; }

        int get_w() const interface(v2) { return w; }

        interface(v3) {
            bool only_for_v3() { return true; }
        }
    }
};

// sizeof(interface(std::cxx20) point) == 12.
// sizeof(interface(v2) point) == 16;
// sizeof(interface(v3) point) == 16;

```

- What if I don't care about C++20 ABI compatibility, and I just want to manage by interface stability going forward?

In that case, you can ignore the standard tags and just add interface declarations as appropriate.


```

interface v1;
interface v2 : v1;
...
struct point {
    interface(v1) {
        int x, y, z;
        interface(v2) int w;

        int get_x() const { return x; }

        int get_w() const interface(v2) { return w; }

        interface(v3) {
            bool only_for_v3() { return true; }
        }
    }
};

// sizeof(interface(v1) point) == 12.
// sizeof(interface(v2) point) == 16;
// sizeof(interface(v3) point) == 16;

```

- What if I'm writing a function that takes standard-library types as parameters, and it might be called by clients using current and future revisions of C++?

Then you'll want to take these unresolved-interface-qualified types.

```

void foo(interface(std::cxx23+) std::vector<int> &x) {
    // We can be called with a C++23 vector, or a C++26 vector, or any later v
}

```

§ 14. ODR and Definition Flexibility

There are some choices around how we adjust our ODR rules and these have design implications for the feature. We will need to adjust our ODR rules to allow for multiple definitions of types which are not exactly the same, but rather, one definition includes entities with interface qualifiers not found in the other.

We might also decide to allow reordering of member declarations within a type while deciding equivalence. Allowing for this kind of reordering may require a more-resilient indexing scheme into the type metadata (e.g., instead of a fixed-index vtable, a hash lookup or binary search might be needed to find member offsets).

We might also decide to allow for renaming of things. This could be done by allowing for the association of a UUID (used, e.g., for mangling instead of the name itself) with entities that are allowed to be renamed.

§ 15. Proposal

§ 15.1. Interface Tags

First, this proposal introduces the concept of an interface tag. An interface tag identifies an interface version and has one or two properties:

1. A name. This name has the usual properties of names in C++: it has a scope, it may be qualified, and so on.
2. Optionally, a parent. An interface may specify an interface from which it inherits. Only single inheritance is permitted (i.e., interface versions have only a linear history).

```
// Declares an interface tag:  
interface v1;  
// Declares an interface tag that inherits from the one above.  
interface v2 : v1;
```

The intent of this proposal is that each version of the standard will define an interface tag associated with that version of the standard, along with the tags defined in previous revisions of the standard, with an appropriate inheritance structure. For example, we might have:

```
namespace std {  
  
    interface cxx20;  
    interface cxx23 : cxx20;  
  
}
```

Note: It is expected that implementations will treat the cxx20 tag as a special case for the purpose of maintaining ABI compatibility with code compiled using tools complying with previous revisions of the C++ standard. Specifically, entities with the cxx20 interface tag may follow different name-mangling rules than other entities.

ISSUE 1 Can interfaces themselves be template arguments and, thus, be dependent?

§ 15.2. Interface Blocks

Recognizing that it is often the case that an interface will apply to everything in large sections of source code, this proposal introduces interface blocks. An interface block can appear at any scope (i.e., namespace scope, class scope, function-scope, etc.) and only cause all declarations and definitions within the interface block to have an implicit interface qualifier with the provided interface tag.

```
namespace foo {  
    interface(v1) {  
        struct x;  
    }  
  
    interface(v2) {  
        struct y;  
        struct x : public y;  
    }  
  
    struct z {  
        interface(v1) {  
            int x;  
        }  
  
        interface(v2) {  
            int x;  
            int y;  
        }  
    };  
}
```

Note: Formally, interface blocks do not introduce new scopes, but are merely syntactic sugar for adding the interface qualifier to everything relevant inside the curly braces. This might seem strange, but in context this makes sense because the interface qualifier itself has a name-scoping-like effect, and so from the programmer's perspective, the construct should have the behavior implied by the braces.

An interface block with an empty interface restores use of the default interface.

§ 15.3. Interface Qualifiers

This proposal introduces a new interface qualifier. There are two kinds of interface qualifiers for:

1. Resolved interfaces - These refer to a specific, named interface.
2. Unresolved interfaces - These refer to a named base interface, and the object might have that interface or any interface (transitively) inheriting from it.

```
// A qualifier for the interface tag v1.  
interface(v1)  
// A qualifier for the interface tag v1 or any interface inheriting from it  
// This is called an unresolved interface.  
interface(v1+)
```

These qualifiers can appear on functions, variables, and so on. Interface qualifiers become part of the type. This has all of the usual properties (e.g., they can be used for overloading, template specialization, and so on).

There is an implementation-defined default interface tag applied to all declarations and definitions. This default must be the same for all entities within a module.

Note: This is to allow the implementation to import modules previously compiled under previous revisions of the C++ standard and have entities in those modules have the "older" interface tag.

An interface qualifier with an empty interface tag specifies that the default interface is to be used.

When performing name lookup, there might be multiple available names in a scope that only differ by their interface tag. In this case, names with an interface tag matching those of the object (for member access), or parameters (analogous to ADL) for any calls, are preferred.

§ 15.3.1. Interface Additivity

Entities cannot have more than one interface, although if an entity has an interface qualifier (explicit or implicit) for both an interface and an interface from which it inherits, the parent interface qualifier is ignored. Otherwise, the program is illformed.

Members of an aggregate must all have interfaces with a common base interface, otherwise the program is illformed.

The set of interface-visible, qualified names associated with an interface tag, including any explicitly-deleted names, must be a superset of the names associated with the tag from which it inherits. Otherwise, the program is illformed. Interface-visible names are those which are namespace scope, public, or used (even in an unevaluated context) by interface-visible entities with inline linkage.

Entities within the same scope with the same name and type (ignoring interface tags) that have storage must be aliases to the same object (i.e., must share that storage).

Note: The above rules are intended to ensure that interface inheritance is strictly additive (although additive in this context does include the ability to explicitly make names unavailable by deleting them) and storage sharing takes place as needed.

Interface qualifiers on fundamental types are ignored. The implementation may define extended fundamental types on which interface qualifiers are ignored. Collectively, these are called interface oblivious types.

Note: The standard library provides a type trait that can be used to determine if interface qualifiers will be ignored for a given base type.

§ 15.3.2. Unresolved Interfaces

Types with unresolved interfaces are distinct from types with resolved interfaces to the same base interface tags.

Note: `dynamic_cast`, and other associated mechanisms, can be used to determine if an object with a given type can be converted to another type (perhaps differing only in the interface qualifier).

Variables with types with unresolved interfaces may hold values of the same base type and any interface tag that (transitively) inherits from the provided base interface or the base interface itself.

Note: The restriction to types with explicitly-provided interface tags is needed to permit types to omit their hidden metadata fields associated with providing interface support.

`offsetof` cannot be applied to types with unresolved interfaces, and neither can `sizeof/alignof`. These can only be used to make pointer/reference types.

Note: The offsets of members within a type with an unresolved interface are not available at compile time, and can be different for different values.

All interface-public members can be accessed from a type with an unresolved interface, but all types on those interfaces will be act as though they're also unresolved. If any of these types do not support use with unresolved interfaces, the program is illformed.

§ 15.4. Header `<stdinterface>`

```
namespace std {
    interface cxx20;
    interface cxx23 : cxx20;

    template<class T>
    struct is_interface_oblivious;

    template<class T>
    inline constexpr bool is_interface_oblivious_v = is_interface_oblivious<T>::value;
}
```

`is_interface_oblivious` can be used to detect if the type is interface oblivious (i.e., ignores interface qualifiers). This is true for fundamental types and may also be true for an implementation-defined set of other implementation-defined types. Otherwise, it is false.

ISSUE 2 Other traits should be provided. A traits to strip off the interface qualifier seems likely needed.

§ 16. Acknowledgments

We thank Nevin Liber, Corentin Jabot, Niall Douglas, Bjarne Stroustrup, Arthur O’Dwyer, Tom Honermann, Gabriel Dos Reis, Bryce Adelstein Lelbach, Richard Smith, Peter Dimov, Peter Bindels, Nathan Myers, Mathias Stearn, Caleb Sunstrum, Stephan T. Lavavej, Ben Craig, Michael Hava, Steve Downey, Davis Herring, Attila Fehér, David Stone, Arvid Norberg, Tony Van Eerd, Bronek Kozicki, and Jon Chesterfield for providing feedback.

This research was supported by the Exascale Computing Project (17-SC-20-SC), a collaborative effort of two U.S. Department of Energy organizations (Office of Science and the National Nuclear Security Administration) responsible for the planning and preparation of a capable exascale ecosystem, including software, applications, hardware, advanced system engineering, and early testbed platforms, in support of the nation’s exascale computing imperative. Additionally, this research used resources of the Argonne Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC02-06CH11357.

§ Issues Index

ISSUE 1 Can interfaces themselves be template arguments and, thus, be dependent? [↩](#)

ISSUE 2 Other traits should be provided. A traits to strip off the interface qualifier seems likely needed. [↩](#)